Windows 환경에서 MCP로 구성하는 AI 기반 모바일 앱 보안 감사

MCP · Tool · Skill · Evidence 기반 운영 구조

Windows에서 Android 앱 보안 감사를 MCP 도구와 Skill 절차로 운영하는 방법을 설명합니다.

작성일: 2026-04-24 · 범위: Android 앱 감사, MCP 연결, Skill 운영, 보안 통제

1. 발표 목표와 범위

발표 목표

- MCP, Tool, Skill의 차이를 명확히 구분
- Windows 환경에서 모바일 앱 감사용 도구체인 구성
- 정적, 동적, 네이티브, 웹 분석 도구의 연결 지점 설명
- AI가 감사를 어떻게 보조할지와 통제 원칙 설명

포함 범위

- Android 앱 보안 감사
- Windows 기반 MCP Host 구성
- Android, JADX, Frida, Ghidra, Chrome DevTools, Burp 계열 도구 연결
- 증거 수집, Finding 초안, 보고서화 흐름

이 발표의 핵심 메시지

AI가 감사를 대체하는 것이 아니라,
MCP로 연결된 도구들을 Skill 기반 절차 안에서 조율해
감사의 재현성, 증거성, 보고서 품질을 높이는 것이 목적이다.

제외 범위

- 무단 공격 자동화
- 비승인 능동 스캔
- 실제 운영 서비스 대상 위험 검증 자동 실행

운영 원칙

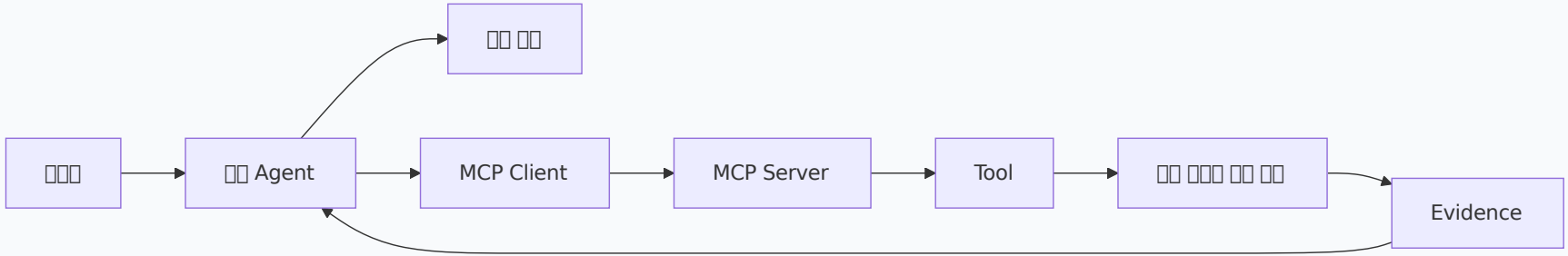
- 승인된 범위만 테스트
- 사람 검토 전 Finding 확정 금지
- 위험 작업은 승인 후 실행

2. Agent · MCP · Tool · Skill

개념을 섞지 않고 보는 것이 중요하다

계층	역할	예시
Agent	목표 해석, 절차 선택, 결과 요약	Claude Code, Codex CLI, Cursor
MCP	Client와 Server를 잇는 연결 규격	JSON-RPC 기반 프로토콜
MCP Server	여러 Tool을 묶어 노출하는 서버	Android MCP, Frida MCP
Tool	실제 실행 단위	`get_screenshot`, `run_script`
Skill	절차와 판단 기준을 담은 플레이북	Mobile Audit Skill

관계 구조



핵심 구분

- MCP는 외부 도구를 연결하는 방식이다
- Tool은 실제로 실행되는 함수다
- Skill은 어떤 순서로 무엇을 검증할지 정하는 절차다
- Agent는 결과를 읽고 다음 행동을 정하는 조정자다

표현도 이렇게 맞추는 편이 정확하다.
"MCP로 스크린샷을 찍는다" 보다는
"Android MCP Server의 스크린샷 Tool을 호출한다"

3. Windows 기반 테스트 랩

기본 구성

- Windows 10/11
- Python 3.10+
- Node.js LTS
- Git
- Android SDK Platform-Tools
- JDK
- Android 디바이스 또는 에뮬레이터

주요 도구

- JADX
- Frida / frida-server
- Ghidra
- Chrome / Chrome DevTools
- Burp Suite 또는 ZAP
- MobSF / Semgrep / CodeQL

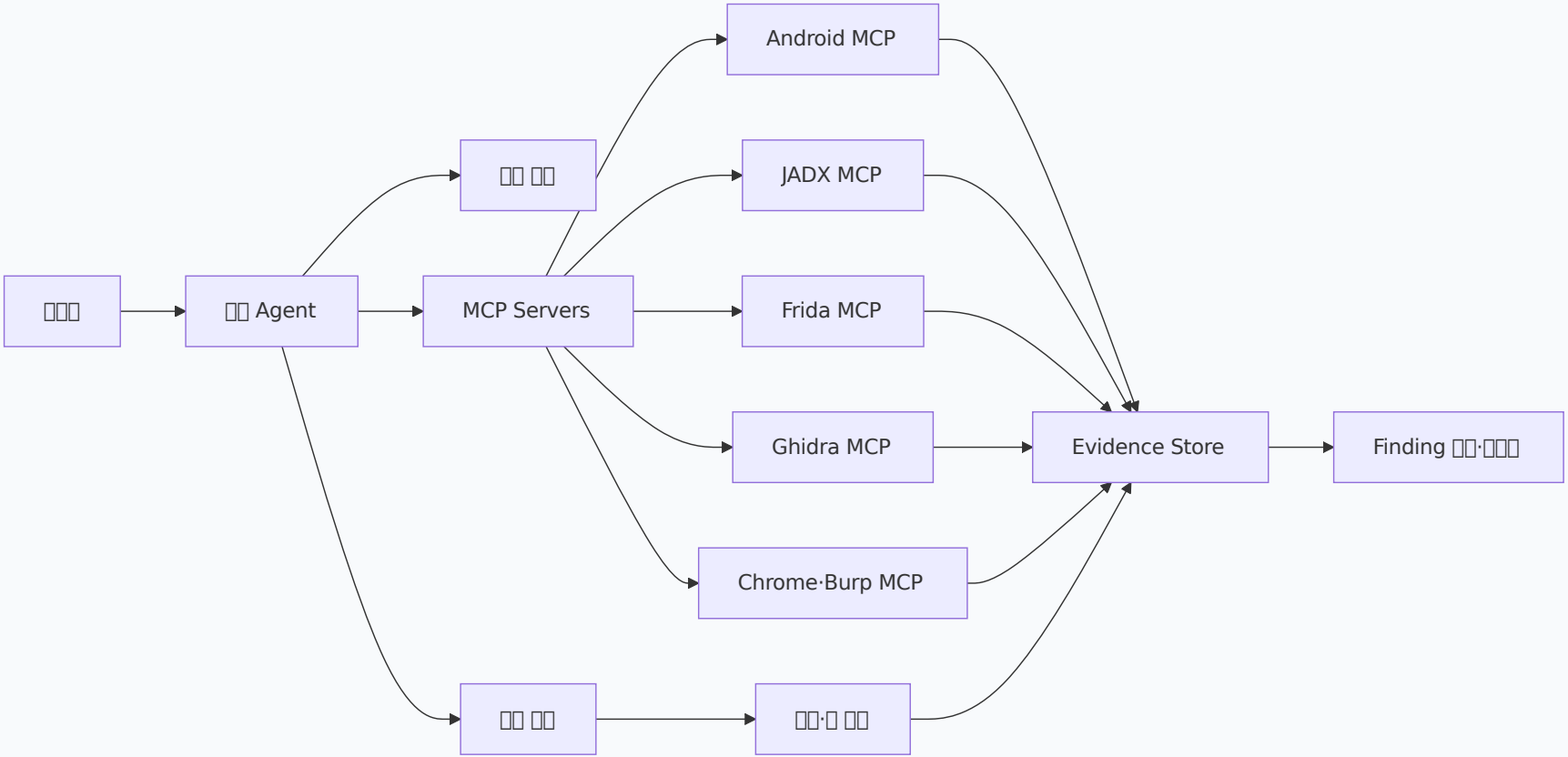
운영 전 확인 항목

```
python --version
node --version
git --version
adb version
adb devices
```

- Windows에서는 경로 표준화가 중요하다
- 도구 설치 루트를 `C:\\Tools` 아래로 정리하면 관리가 쉽다
- 버전과 테스트 디바이스 정보를 기록해 두어야 재현성이 유지된다

4. 전체 아키텍처

AI 기반 모바일 감사 구조



왜 이 구조가 필요한가

- 모바일 앱 감사는 단일 도구로 끝나지 않는다
- 정적 분석, 동적 분석, 네트워크 분석, 네이티브 분석이 연결되어야 한다
- 결과를 한 번에 보고서로 만들려면 증거 저장 구조가 먼저 있어야 한다
- Agent는 여러 결과를 이어 붙여 Finding 후보를 만드는 데 강하다

5. Android MCP

Android MCP의 역할

- 디바이스 정보 조회
- 스크린샷 수집
- UI Layout XML 수집
- Logcat 수집과 필터링
- 패키지 정보 및 권한 조회
- 파일 pull, push 같은 증거 보조 작업

대표 Tool	감사 활용
`get_device_info`	디바이스 기준선 확보
`get_screenshot`	현재 화면 증거 수집
`get_uilayout`	UI 요소 구조 분석
`get_logcat`	인증, 오류, 보안 이벤트 확인

적용 포인트

```
adb devices
adb shell getprop ro.product.model
adb shell pm list packages
```

- ADB 연결 안정화가 먼저다
- 디바이스가 여러 개면 serial 지정이 필요하다
- 설치, 삭제, shell 실행은 승인 게이트 뒤에 두는 편이 안전하다
- 로그인 화면 캡처, 버튼 좌표 계산, 인증 로그 수집 같은 반복 작업에 특히 유용하다

6. JADX와 정적 분석

JADX MCP가 주는 가치

- APK 디컴파일 코드 탐색
- AndroidManifest 분석
- 권한과 exported component 식별
- 하드코딩 Secret 후보 탐지
- WebView, 저장소, 인증 흐름 후보 찾기

함께 쓰기 좋은 도구

- Semgrep
- MobSF
- CodeQL
- 수동 코드 리뷰

정적 분석 결과는 후보군이다

분석 결과	바로 Finding인가
하드코딩 키 문자열	아님, 실제 키 성격 검증 필요
Exported Activity	아님, 인증과 권한 체크 확인 필요
`setJavaScriptEnabled(true)`	아님, 실제 노출면 확인 필요
평문 저장 함수	아님, 저장 경로와 데이터 성격 확인 필요

정적 분석의 역할은 **깊게 볼 위치를 좁히는 것**이다. 그 다음 단계에서 Android MCP, Frida, Burp 결과와 연결한다.

7. Frida와 런타임 검증

Frida MCP의 역할

- 함수 호출 인자와 반환값 관찰
- 런타임에 복호화되는 값 확인
- 인증 흐름과 암호화 흐름 추적
- 난독화된 코드의 실제 동작 확인
- 네이티브 함수 호출과 Java bridge 연결 보조

동적 분석에서 중요한 점

- 정적 분석에서 본 후보를 검증하기 위한 도구로 사용
- 테스트 계정과 승인된 환경만 사용
- 토큰, 키, 비밀번호는 마스킹한 메타데이터만 남김
- 후킹 스크립트 실행 전 대상 함수와 클래스명을 검토

```
예시 요청:
LoginManager.validateUser 호출 인자와 반환값을 추적하고
성공/실패 흐름만 짧게 요약해줘.
민감값 원문은 출력하지 마.
```

8. Ghidra와 네이티브 분석

Ghidra MCP가 필요한 경우

- APK 안의 `.so` 라이브러리를 분석해야 할 때
- JNI 함수와 Java bridge를 확인해야 할 때
- 암호화, 문자열 복호화, 안티디버깅 로직을 봐야 할 때
- 의미 없는 함수명을 분석 목적에 맞게 정리해야 할 때

실무적 가치

- JADX로 보이지 않는 로직을 확인할 수 있다
- Frida 후킹 대상 함수를 선정하는 데 도움이 된다
- 설계상 민감 로직이 클라이언트에 있는지 판단할 수 있다

예시 질문	기대 결과
JNI 함수가 어떤 데이터를 반환하는가	복호화, 조합, 하드코딩 여부 파악
안티디버깅 흔적이 있는가	`ptrace`, `TracerPid` 등 후보 정리
함수명을 바꿔도 되는가	리네이밍 제안만 하고 사람 검토 후 반영

9. WebView · 브라우저 · 네트워크

Chrome DevTools, Playwright, Burp 계열 도구

- WebView DOM 구조 분석
- 로그인과 브라우저 플로우 자동화
- Network 요청, 응답, 헤더, 저장소 확인
- 콘솔 에러, CSP, Mixed Content 확인
- API 요청과 코드 위치를 연결해 해석

강점

- WebView와 API 보안을 모바일 감사 흐름에 통합
- Storage, Cookie, LocalStorage 위험을 빠르게 확인
- 로그인 플로우를 재현 가능하게 기록

주의점

- 페이지 콘텐츠를 모델 지시문처럼 신뢰하면 안 된다
- 프롬프트 인젝션 가능성 고려
- 토큰, 쿠키, 개인정보는 마스킹
- 능동 스캔은 승인된 범위만 수행

10. 자동 스캐닝과 초벌 분류

MobSF · Semgrep · CodeQL의 위치

- MobSF는 APK 초벌 정적 분석에 강하다
- Semgrep은 반복 가능한 룰 기반 점검에 좋다
- CodeQL은 데이터플로우 기반 검증에 적합하다

도구	역할	주의점
MobSF	초벌 요약, Manifest, 설정 점검	결과를 그대로 보고서에 넣지 않기
Semgrep	커스텀 룰 기반 후보 탐지	오탐과 중복 제거 필요
CodeQL	소스 기반 고급 질의	소스와 쿼리 품질 의존

핵심 원칙

자동 스캐닝 도구는 **Finding 확정 도구가 아니라 후보 생성 도구**다. AI는 이 후보를 묶고 우선순위를 정리하는 데 도움을 준다.

11. AI Mobile Audit Orchestrator

단계별 실행 흐름

단계와 산출물

단계	주 도구	산출물
대상 등록	Agent, Scope 문서	앱 정보, 범위
기본 정보 수집	Android MCP	디바이스, 패키지, 화면
정적 분석	JADX, Semgrep, MobSF	코드 위치, 후보군
동적 분석	Frida, Android MCP	함수 로그, 실행 증거
네이티브 분석	Ghidra	JNI, 암호화, 안티디버깅 후보
웹/API 분석	Chrome, Burp	요청/응답, 저장소, 콘솔
보고서화	Skill Template	Finding 초안, MASVS 매핑

12. Skill과 Evidence Bundle

Skill은 절차를 정한다

- 단계별 도구 선택 기준
- 승인 필요 작업 정의
- 체크리스트와 판단 기준 정리
- Finding 템플릿과 보고서 형식 제공
- 증거 저장 규칙 통일

Evidence Bundle은 결과를 남긴다

- 스크린샷
- Logcat
- HTTP 요청과 응답
- 코드 위치와 클래스명
- 동적 후킹 로그
- MASVS 매핑 근거

표준 저장 구조 예시

```
evidence/{app}/{date}/
  01_device/
  02_static/
  03_dynamic/
  04_native/
  05_network/
  06_findings/
```

- 증거가 표준화되어야 재검증이 쉬워진다
- Finding에는 증거 경로와 재현 절차가 함께 있어야 한다
- AI는 초안을 정리하고, 사람은 영향도와 확정 여부를 판단한다

13. 운영 통제

반드시 통제해야 할 것

- 과도한 Tool 권한
- 셸 명령 오남용
- 민감정보의 모델 컨텍스트 유입
- 웹 페이지와 이슈 본문 기반 프롬프트 인젝션
- 악성 MCP Server 설치
- 승인 없는 능동 스캔

권장 통제 원칙

영역	통제 방식
대상 범위	allow-list 기반
위험 작업	승인 게이트
비밀값	자동 redaction
도구 권한	최소 권한 토큰
로그	Tool Call audit log 보존
실행 환경	테스트 환경 격리
결과 확정	Human-in-the-loop

14. 결론

한 문장으로 정리하면

MCP는 연결, Tool은 실행, Skill은 절차, Auditor는 판단이다.

최종 요약

- MCP는 다양한 보안 도구를 일관된 방식으로 연결한다
- Tool은 실제 실행 단위이며 권한 통제가 필요하다
- Skill은 감사 절차와 산출물 품질을 표준화한다
- Agent는 여러 증거를 엮어 Finding 후보를 만든다
- 최종 검증과 책임은 감사자에게 남아 있어야 한다

Q&A

설치, 권한 통제, 팀 운영 방식, Finding 검증 절차를 중심으로 이어서 논의합니다.

15. 참고 자료

공식 문서와 주요 도구

- Model Context Protocol Specification
- MCP Basic Concepts
- Claude Code MCP Documentation
- Claude Code Skills Documentation
- OpenAI Codex MCP Documentation
- OpenAI Codex Configuration Reference
- OWASP MASVS / MASTG

도구별 참고 링크

- Android MCP Server
- JADX MCP Server
- JADX AI MCP
- Frida MCP
- GhidraMCP
- Chrome DevTools MCP
- Playwright MCP
- Semgrep MCP
- CodeQL Development MCP Server
- MobSF
- GitHub MCP Server